

Comparison Operators

Operator	Meaning
`<` `>`	< >
`<=` `>=`	≤ ≥

`==` | = | | `!=` | ≠ |

Warning: `=` is used for assignment in Python. For equality check use: `==`

The result of a comparision is always one of the two special objects: `True` or `False`

In [73]:
```
12 > 1, 12 >= 1, 12 <= 1, 12 == 1, 12 != 1
```

Out[73]:
```
(True, True, False, False, True)
```

`not` makes `True` → `False` and vice versa. `not` has lower precedence than the comparision operators.

In [74]:
```
not 12 > 1
```

Out[74]:
```
False
```

Since `True` and `False` are also some Python objects they can be compared as well.

In [ ]:
```
(12 != 1) == (12 == 12)
```

```
True
```

Compound Comparisions

In Python you can combine multiple comparisions in a single expression:

In [76]:
```
1 < 12 < 30 > 1 <= 100
```

Out[76]:
```
True
```

Warning: The previous example is different than the following example.

In [77]:
```
1 < 12 < (30 > 1) <= 100
```

Out[77]:
```
False
```

Because when you use paranthesis, Python will evaluate the expression between paranthesis first. So `(30 > 1)` will become `True`. Then the comparision will be done like `1 < 12 < True <= 100`.

Alternatively, you can use:

- `and` operator for ∧ in mathematics
- `or` operator for ∨ in mathematics

a	b	a and b	a or b
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

to combine multiple comparisions. The precendence of all arithmetic operators and bitwise operators are higher than `not` operator then `and` operator then `or` operator.

In [79]:
```
1 < 12 and 12 < 30 and 30 > 1 and 1 <= 100 or not True
```

Out[79]: True

is equivcalent to:

In [80]:
```
((1 < 12) and (12 < 30) and (30 > 1) and (1 <= 100)) or (not True)
```

Out[80]: True

Sorting The Characters

Since the characters have order then they can be compared as well.

In [81]:
```
"!" < "1" < "A" < "B" < "a" <= "b"
```

Out[81]: True

Warning: But you can not compare a character with a number.

Comparing Strings

In [ ]:
```
"hello world!" == 'hello world!'
```

In [ ]:
```
'hello world!' > 'hello world!'
```

Here the comparision is done in *lexicographic* order. i.e.

- Initially, Python compares the first characters of both strings.
- If the first characters of both strings are same then the comparission is done between the second characters.
- If the second characters are also same then the third.
- And so on.

Comparing Tuples

In [4]:
```
('hello', 'world!') > ('hello',)
```

Out[4]: True

Here the comparision is done in *lexicographic* order.